

Module Guide for Software Engineering

Team 11, technically functional

Matthew Huynh

Cieran Diebolt

Vaisnavi Shanthamoorthy

Maham Siddiqui

Eman Ashraf

November 13, 2025

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
Software Engineering	Explanation of program name
UC	Unlikely Change

Contents

1	Revision History	i
2	Reference Material	ii
2.1	Abbreviations and Acronyms	ii
3	Introduction	1
4	Anticipated and Unlikely Changes	2
4.1	Anticipated Changes	2
4.2	Unlikely Changes	2
5	Module Hierarchy	3
6	Connection Between Requirements and Design	3
7	Module Decomposition	5
7.1	Hardware Hiding Modules	5
7.2	Behaviour-Hiding Modules	5
7.2.1	Authentication Module (M2)	5
7.2.2	User Account Management Module (M3)	6
7.2.3	Organization Linking Module (M4)	6
7.2.4	Exercise Library Module (M5)	6
7.2.5	Exercise Program Management Module (M6)	7
7.2.6	Video Capture Module (M7)	7
7.2.7	Video Storage Module (M8)	7
7.2.8	Feedback Generation Module (M11)	8
7.2.9	Reporting Module (M12)	8
7.2.10	Dashboard Module (M13)	8
7.3	Software Decision Module	9
7.3.1	Database Management Module (M5)	9
7.3.2	Generator Module (M6)	9
7.3.3	Metrics Module (M7)	10
7.3.4	Analyze Module (M8)	10
8	Traceability Matrix	11
9	Use Hierarchy Between Modules	12
10	User Interfaces	12
11	Design of Communication Protocols	12
12	Timeline	12

List of Tables

1	Module Hierarchy	4
2	Trace Between Requirements and Modules	11
3	Trace Between Anticipated Changes and Modules	11

List of Figures

1	Use hierarchy among modules	12
---	---------------------------------------	----

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the “secrets” that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module’s data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers’ understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The user interface of the application may be changed to include additional features for accessibility. Changes include:

1. How feedback is presented to the user as the exercise is performed (i.e. written, auditory, alerts, etc.).
2. How the insights are generated to the user on their account dashboard (i.e. displayed as a line graph, columns of data, does the user have the option to select from multiple views, etc.).

AC2: The angles that are calculated to support different exercises. Currently, the application supports the 'single leg raise' exercises which is routinely used in rehabilitation for post-operative knee surgery. Future iterations of the application may include additional body parts and exercises.

AC3: Changes to the feedback/guidance given. As more requirements are met, there should be changes to encapsulate the prompts required for the motion (i.e. 'keep the torso straight as this movement is performed'). Additionally, there may be alternate prompts with varying degrees of complexity, which relates to accessibility and the patients understanding of the exercise (i.e. 'during this motion you want to engage your quadriceps as you drive your toe towards the sky 8 inches off the ground').

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: Recording/Video Upload, Output: On-Screen Feedback/Corrections).

- UC2:** The knee will remain modeled as a single-degree-of-freedom (1-DOF) hinge joint in the sagittal plane (flexion and extension only). No expansion to multi-DOF modelling or other joints is planned.
- UC3:** Users will only perform physiotherapist-approved rehabilitation exercises. The system will not automatically generate or recommend new, unverified exercises.
- UC4:** Feedback will be limited to movement-based corrections and joint angle guidance. The system will not provide medical diagnosis or replace professional physiotherapy assessment.
- UC5:** The system is not being built to integrate feedback from external sensors, wearable devices, and specialized specialized motion-capture equipment.
- UC6:** The scope will remain focused on knee rehabilitation. The system will not expand to whole-body tracking or unrelated physiotherapy movements.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: User Interface Module

M2: Home Module

M3: Main Module

M4: Draw Module

M5: Database Management Module

M6: Generator Module

M7: Metrics Module

M8: Analyze Module

M9: Feedback Module

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

Level 1	Level 2
Hardware-Hiding Module	
Behaviour-Hiding Module	User Interface Module Home Module Main Module Draw Module
Software Decision Module	Database Management Module Generator Module Metrics Module Analyze Module Feedback Module

Table 1: Module Hierarchy

Design Decisions

Security To satisfy security and privacy requirements (FR21–FR23, SR-IR1, SR-PR1–2), the system implements end-to-end encryption for data at rest and in transit using industry-standard encryption protocols. User authentication employs session-based tokens with automatic timeout after 60 minutes of inactivity.

Real-time Performance To meet the requirement for real-time feedback (FR13–14, PR-SL1–2), the computer vision processing pipeline is optimized to provide analysis results within 1 second of detecting form deviations. This is achieved through efficient pose estimation algorithms and selective processing of critical joint angles.

Offline Capability To address reliability requirements (PR-RFT2, MS-AD1), the system implements local caching of exercise instructions and video queuing for uploads. This ensures users can access exercise information and record sessions even without internet connectivity, with automatic synchronization when connection is restored.

Modular Exercise Library To support extensibility (PR-CR1, PR-SER1), the exercise library is designed as a modular system where new exercises and joint types can be added without modifying core analysis logic. Each exercise type has its own parameter set defining acceptable ranges and form criteria.

Two-Camera View Analysis To address the risk of 3D motion measurement with 2D cameras, the system supports analysis from the sagittal view as primary, with optional frontal view for enhanced accuracy. The analysis algorithms are designed to work with single or multiple camera angles.

7 Module Decomposition

Modules are decomposed according to the principle of “information hiding.” The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Software Engineering* means the module will be implemented by the Software Engineering software. Only the leaf modules in the hierarchy have to be implemented.

7.1 Hardware Hiding Modules

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software, including camera access, screen display, audio output, and device sensors. The system can use it to capture video, display outputs, or accept touch inputs.

Implemented By: Android OS

7.2 Behaviour-Hiding Modules

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: Software Engineering

7.2.1 Authentication Module (M2)

Secrets: The implementation details of user authentication, session management, and credential verification.

Services: Handles user login, logout, session creation, and session timeout. Verifies user credentials against stored records and manages authentication tokens. Enforces 60-minute inactivity timeout as per SR-AR3.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR1, FR1.1, SR-AR1, SR-AR3

7.2.2 User Account Management Module (M3)

Secrets: The structure and validation logic for user account data, including patient and physiotherapist profiles.

Services: Creates and manages user accounts for both patients and physiotherapists. Validates registration data including government ID, license numbers (for PTs), and invite codes (for patients). Stores and retrieves user profile information securely.

Implemented By: Software Engineering

Type of Module: Abstract Data Type

Related Requirements: FR1.1, FR1.2, FR1.3, FR2

7.2.3 Organization Linking Module (M4)

Secrets: The algorithm for generating, validating, and managing access codes that link patients to physiotherapists.

Services: Generates unique access codes for physiotherapists to share with patients. Validates patient-submitted access codes and establishes the patient-PT relationship. Manages the patient list for each physiotherapist. Handles patient account deletion requests from PTs.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR1.3, FR2, FR3, FR18

7.2.4 Exercise Library Module (M5)

Secrets: The storage structure and retrieval methods for exercise information, including instructions, media, and form parameters.

Services: Maintains a library of at least ten knee exercises with detailed instructions, demonstration media, and form parameters. Provides exercise information to users on demand. Supports caching for offline access.

Implemented By: Software Engineering

Type of Module: Abstract Data Type

Related Requirements: FR5, MS-AD1

7.2.5 Exercise Program Management Module (M6)

Secrets: The data structures and business logic for managing patient-specific exercise programs, including customization and progress tracking.

Services: Allows physiotherapists to create and customize exercise programs for individual patients, including sets, reps, rest times, and weekly limits. Tracks patient exercise completion and progress over time. Enforces PT-defined limitations on exercise frequency and intensity.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR6, FR10, FR11, FR22, FR27

7.2.6 Video Capture Module (M7)

Secrets: The implementation of video recording, including camera configuration, frame rate management, and recording controls.

Services: Captures video of users performing exercises at minimum 720p resolution and 30 fps. Performs pre-capture environment checks (lighting, framing). Provides recording controls (start, stop, retry). Ensures video meets minimum quality standards before proceeding to analysis.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR7, FR15, FR26, UHR-EOU3

7.2.7 Video Storage Module (M8)

Secrets: The mechanism for storing, retrieving, and managing video files, including local caching and cloud synchronization.

Services: Stores recorded exercise videos securely (encrypted at rest per SR-IR1). Manages local storage and cloud upload queuing. Handles upload retry logic when network is unavailable. Retrieves stored videos for patient and PT review. Implements automatic backup functionality.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR21, SR-IR1, PR-RFT2, MS-MNT1

7.2.8 Feedback Generation Module (M11)

Secrets: The rules and logic for converting analysis results into user-friendly feedback messages and instructions.

Services: Generates real-time corrective feedback messages based on form analysis results. Provides step-by-step exercise guidance. Creates descriptive messages for form deviations specifying required adjustments. Triggers safety warnings for dangerous movements.

Implemented By: Software Engineering

Type of Module: Library

Related Requirements: FR13, FR14, FR16, PR-SCR1

7.2.9 Reporting Module (M12)

Secrets: The format and generation logic for exercise session reports and progress summaries.

Services: Generates comprehensive session reports including completion rate, form score, pain level, and patient feedback. Creates progress summaries showing improvement over time. Generates alerts for repeated incorrect attempts or concerning patterns. Formats reports for PT review.

Implemented By: Software Engineering

Type of Module: Library

Related Requirements: FR17, FR19, FR20, FR25

7.2.10 Dashboard Module (M13)

Secrets: The user interface layout and interaction logic for patient and physiotherapist dashboards.

Services: Displays patient exercise history, progress charts, and upcoming exercises. Shows PT patient list with status indicators and alerts. Provides access to video review and report viewing. Enables PT comments on patient sessions. Presents analytics and visualizations of exercise data.

Implemented By: Software Engineering

Type of Module: Abstract Object

Related Requirements: FR18, FR19, FR24, FR25

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are not described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: PhysioCompanion

7.3.1 Database Management Module (M5)

Secrets: The structure, validation, and authorization logic for all user credentials, account data, invite codes, physiotherapist identification numbers, exercise information, and data encryption.

Services: Handles user login, logout, session creation, and session timeout. Verifies user credentials against stored records and manages authentication tokens. Enforces 60-minute inactivity timeout as per SR-AR3. Creates and manages user accounts for both patients and physiotherapists. Validates registration data including government ID, license numbers (for PTs), and invite codes (for patients). Stores and retrieves user profile information securely. Generates unique access codes for physiotherapists to share with patients. Validates patient-submitted access codes and establishes the patient-PT relationship. Manages the patient list for each physiotherapist. Handles patient account deletion requests from PTs. Maintains a library of at least 10 knee exercises with detailed instructions, demonstration media, and form parameters. Provides exercise information to users on demand. Supports caching for offline access. Encrypts patient data at rest and in transit using industry-standard protocols. Manages encryption keys securely. Provides methods for encrypting and decrypting data as needed by other modules. Ensures compliance with healthcare data security requirements. Enforces role-based access control for patients and physiotherapists. Ensures patients can only access their own data. Ensures PTs can only access data for patients in their patient list. Logs unauthorized access attempts. Validates user permissions before allowing data operations.

Implemented By: Physiotherapy Companion

Type of Module: Abstract Object

Related Requirements: FR1, FR1.1, FR1.2, FR1.3, FR2, FR3, FR5, FR18, FR21, FR23, SR-AR1, SR-AR3, SR-IR1, SR-PR1, SR-PR2, MS-AD1

7.3.2 Generator Module (M6)

Secrets: The format and generation logic for exercise session reports and progress summaries.

Services: Generates comprehensive session reports including completion rate, form score, pain level, and patient feedback. Creates progress summaries showing improvement over time. Generates alerts for repeated incorrect attempts or concerning patterns. Formats reports for PT review.

Implemented By: Physiotherapy Companion

Type of Module: Library

Related Requirements: FR17, FR19, FR20, FR25

7.3.3 Metrics Module (M7)

Secrets: The specific computer vision algorithms and implementation details for pose detection, joint tracking, and body landmark identification.

Services: Processes video frames to detect and track body landmarks using pose estimation. Identifies key joints relevant to knee exercises (hip, knee, ankle, foot). Extracts coordinate data for joint positions across video frames. Handles varying lighting conditions and camera angles. Implements the core CV library integration (OpenCV, MediaPipe).

Implemented By: Physiotherapy Companion

Type of Module: Abstract Data Type

Related Requirements: FR8, FR12

7.3.4 Analyze Module (M8)

Secrets: The mathematical models and algorithms for calculating joint angles, range of motion, and determining form correctness based on exercise parameters. Also the mechanism for storing video files through local caching.

Services: Calculates joint angles from landmark coordinates with $\pm 5^\circ$ accuracy. Measures range of motion with $\pm 10\%$ accuracy. Compares measured values against exercise-specific acceptable ranges. Detects form deviations and classifies severity. Identifies dangerous movements requiring immediate halt. Measures time-under-tension and exercise duration. Also stores recorded exercise videos securely (encrypted at rest per SR-IR1).

Implemented By: Physiotherapy Companion

Type of Module: Abstract Data Type

Related Requirements: FR12, FR13, FR14, FR21, SR-IR1, PR-RFT2, PR-PAR1, PR-PAR2, PR-SCR1, MS-MNT1

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
R1	M9, M??, M??, M??
R2	M??, M??
R3	M??
R4	M??, M??
R5	M??, M??, M??, M??, M??, M??
R6	M??, M??, M??, M??, M??, M??
R7	M??, M??, M??, M??, M??
R8	M??, M??, M??, M??, M??
R9	M??
R10	M??, M??, M??
R11	M??, M??, M??, M??

Table 2: Trace Between Requirements and Modules

AC	Modules
AC??	M9
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

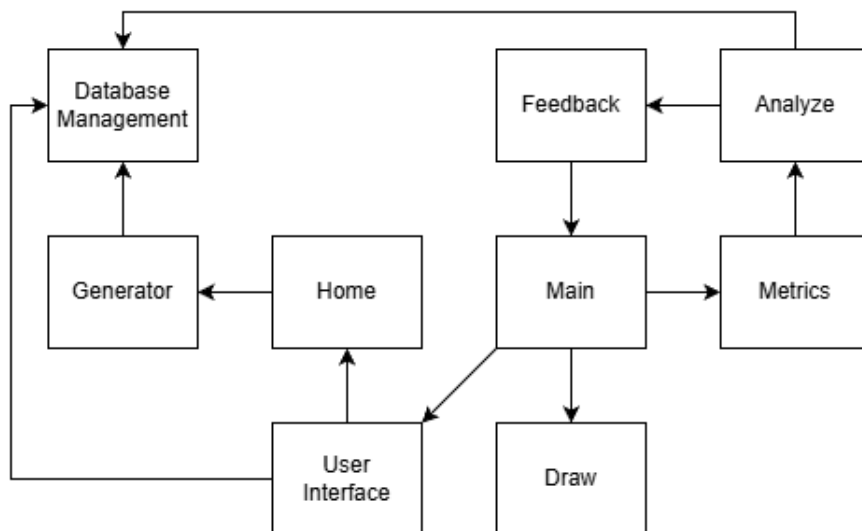


Figure 1: Use hierarchy among modules

10 User Interfaces

The proposed user interface for the system will attempt to model [this Figma prototype](#).

11 Design of Communication Protocols

Communication between the system and the remote database is handled through the Database Management Module. The module will utilize the API methods of the remote database for the purpose of storing and retrieving data. It is through the database that user communication via comments on past activities will occur.

12 Timeline

For the remainder of the term, the project timeline will follow the official course deliverables and our internal task assignments. A detailed task breakdown is maintained on our [GitHub Project Board](#). The high-level schedule is listed below:

Present to Mid-November

- **POC Finalization & Testing** (Responsible: Entire Team)
 - Complete camera and pose detection reliability.
 - Work out demo to showcase to professor and TA

- **Design Document Revision 0** (Responsible: Entire Team)

- Write system architecture, data flow, MediaPipe integration, hazards and mitigations.
- Add code comments, diagrams, and updated requirement traceability.

December to January

- **Work on project as a whole in preposition for Revision 0 Demo** (Responsible: Entire Team)

- Work on entire project, start building and revising the remaining modules, to begin putting together all the pieces

February

- **Rev0 Presentations** (Responsible: Entire Team)

- Implement feedback from the professor and TA in the previous deliverables.
- Refactor code to ensure it is functional, stable, and ready for the presentation.
- Finalize demonstration plan and speaking roles.

Late March – Early April

- **Final Demonstration (Revision 1)** (Responsible: Entire Team)

- Implement improvements from TA feedback.
- Enhance UI transparency (tracking visualization, angles, notifications).
- Prepare live demo and video recording.

- **EXPO Demonstration** (Responsible: Entire Team)

- Build poster and prepare speaking roles.
- Conduct demo rehearsals and ensure hardware stability.

April – Final Week

- **Final Documentation (Revision 1)** (Responsible: Entire Team)

- Submit Problem Statement, Development Plan, POC Plan, Requirements, Hazard Analysis, Design Document, V&V Plan, V&V Report, Extra Documentation, and Source Code.
- Include GitHub repository link, README, demo video, and traceability matrices.

References

- David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.
- D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.